

COMPLEXITY ANALYSIS - USEI14 (Maximum Flow - Edmonds-Karp)

1. INTRODUCTION

USEI14: Maximum Throughput Between Two Hubs

This document provides a detailed temporal complexity analysis of the Maximum Flow algorithm implementation using the Edmonds-Karp approach (Ford-Fulkerson with BFS) for computing maximum railway throughput between two stations in the Belgian railway network.

Key Implementation Details:

- Algorithm: Edmonds-Karp (BFS-based Ford-Fulkerson)
- Graph Representation: Adjacency list (MapGraph)
- Data Structures: HashMap (parent map), HashSet (visited), Queue (BFS)

2. ALGORITHM OVERVIEW

2.1 Core Steps

- Clone original graph (preserve initial capacities)
- Create residual graph (will be modified)
- WHILE augmenting path exists from source to sink:
 - Find augmenting path using BFS
 - Find bottleneck (minimum capacity on path)
 - Update residual capacities (forward and reverse edges)
 - Accumulate flow
- Calculate Max Flow Graph = Initial Graph - Residual Graph

2.2 Key Methods Implemented

Method	Purpose	Complexity
bfsMaxFlow()	Find augmenting path with remaining capacity	O(E)
cloneGraph()	Deep copy graph for residual graph	O(V+E)

Method	Purpose	Complexity
edmondsKarp()	Main algorithm orchestration	$O(V \cdot E^2)$
calculateMaxFlowGraph()	Compute flow that passed through edges	$O(E)$

3. DETAILED COMPLEXITY ANALYSIS

3.1 Method: bfsMaxFlow() [Algorithms.java]

Purpose: Find augmenting path using breadth-first search, considering only edges with remaining capacity > 0 .

Implementation:

Input: residual graph g , source, sink, parent map

Output: true if path exists, false otherwise

1. Create visited set and queue $O(1)$
2. Clear parent map $O(V)$
3. Initialize: add source to queue $O(1)$
4. WHILE queue not empty Loop: $\leq V$ iterations
 - a. Poll node from queue $O(1)$
 - b. For each adjacent node Total: $O(E)$ across all iterations
 - Check if unvisited and has capacity $O(1)$
 - Mark visited and add to queue $O(1)$
 - Early return if sink reached $O(1)$
5. Return false if no path $O(1)$

Time Complexity Breakdown:

- Initialization: $O(1) + O(V) = O(V)$
- Main loop: Visits each vertex once = $O(V)$
- Inner loop: Traverses each edge once = $O(E)$

- Total: $O(V + E) = O(E)$ (since $E \geq V-1$ in connected graphs)

Space Complexity: $O(V)$ for visited set, queue, and parent map

3.2 Method: cloneGraph() [Algorithms.java]

Purpose: Create deep copy of graph for residual graph creation.

Implementation:

Input: original graph

Output: cloned graph with same structure and capacities

1. Create new MapGraph $O(1)$
2. For each vertex in original Loop: V iterations
 - a. Add vertex to clone $O(1)$ each
3. For each edge in original Loop: E iterations
 - a. Get edge properties (cost, distance, cap) $O(1)$ each
 - b. Add edge to clone $O(1)$ each

Time Complexity Breakdown:

- Vertex copying: $O(V)$
- Edge copying: $O(E)$
- Total: $O(V + E) = O(E)$ (since $E \geq V-1$)

Space Complexity: $O(V + E)$ for new graph structure

3.3 Method: edmondsKarp() [MaxFlowAlgorithm.java]

Main Algorithm - Most Critical for Overall Complexity

High-Level Steps:

1. Clone original graph twice $O(2E) = O(E)$
2. Initialize variables (maxFlow, paths, etc) $O(1)$
3. WHILE bfsMaxFlow finds augmenting path Main loop
 - a. Reconstruct path from parent map $O(P)$ - P = path length

- b. Find bottleneck capacity $O(P)$
- c. Update forward edge capacities $O(P)$
- d. Update reverse edge capacities $O(P)$
- e. Accumulate to maxFlow $O(1)$
- f. Clear parent map $O(V)$
- 4. Calculate Max Flow Graph $O(E)$
- 5. Return result $O(1)$

Detailed Complexity Analysis:

The critical insight is the number of times the main loop executes:

Lemma (Edmonds-Karp Property):

- The shortest path from source to sink in the residual graph increases monotonically with iterations
- Maximum distance $\leq V$
- For each distance d , there are at most $O(E)$ iterations
- Total iterations: $O(V \times E)$

Calculation:

Main loop iterations: $O(V \times E)$

Per iteration cost:

- bfsMaxFlow(): $O(E)$
- Reconstruct path: $O(V)$ - maximum path length
- Update capacities: $O(V)$
- Clear map: $O(V)$
- Per iteration: $O(E)$ [BFS dominates]

Total: $O(V \times E)$ iterations $\times O(E)$ per iteration = $O(V \times E^2)$

Final Time Complexity: $O(V \times E^2)$

Space Complexity Breakdown:

Graphs (original + residual): $O(V + E)$ each = $O(V + E)$

Parent map: $O(V)$

Visited set (in BFS): $O(V)$

Queue (in BFS): $O(V)$

Augmenting paths list: $O(V \times E)$ worst case

[$V \times E$ paths $\times V$ nodes each]

Final Space Complexity: $O(V \times E)$ - dominated by paths list